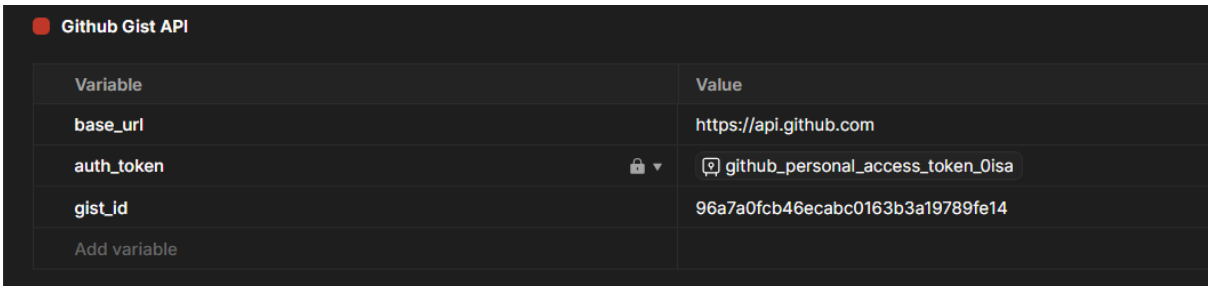
# Exploring and Testing APIs with Postman

GitHub Repository: https://github.com/Elizabeth-Ater/API_Testing

## Postman Environment Setup

### Explanation

The Postman environment contains the variables required for the API requests. These variables allow the same values to be reused across different requests and make the collection easier to manage.




## Pre-Request Script

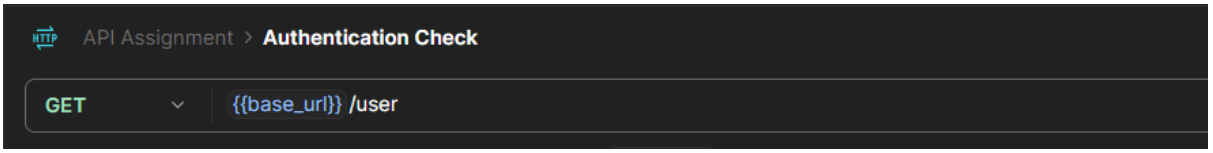
This project did not use a pre-request script, since GitHub Personal Access Tokens do not require refreshing during a session like OAuth2 tokens do. Although the token can expire, this requires manually generating a new one in GitHub's developer settings, not an automatic refresh. The token is stored and reused for all requests through Bearer Token authorization.

## Authentication Check

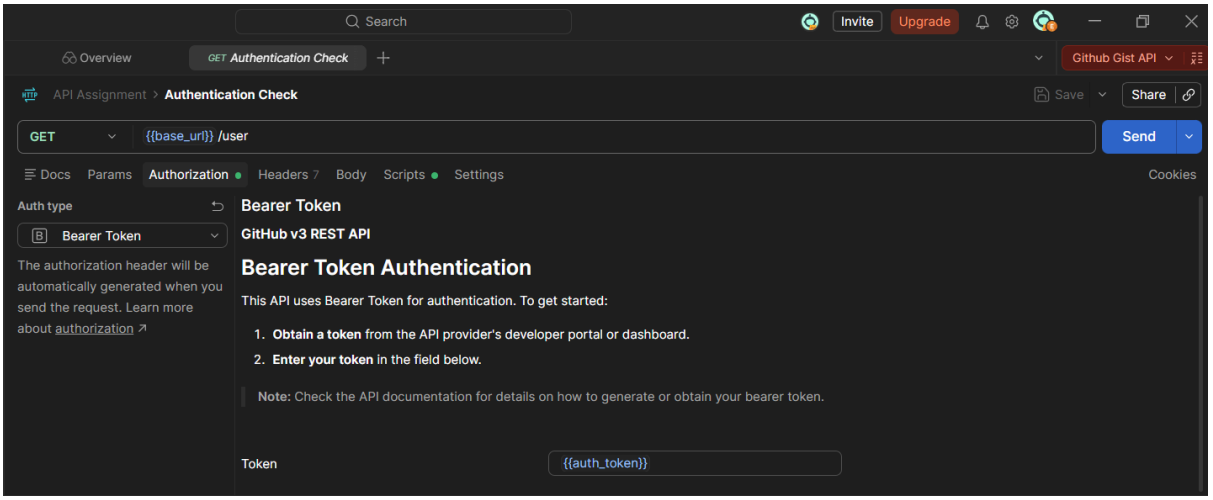
### Explanation

This request checks whether the GitHub Personal Access Token is valid. A successful response confirms that the request is authenticated and that an authenticated GitHub user exists.

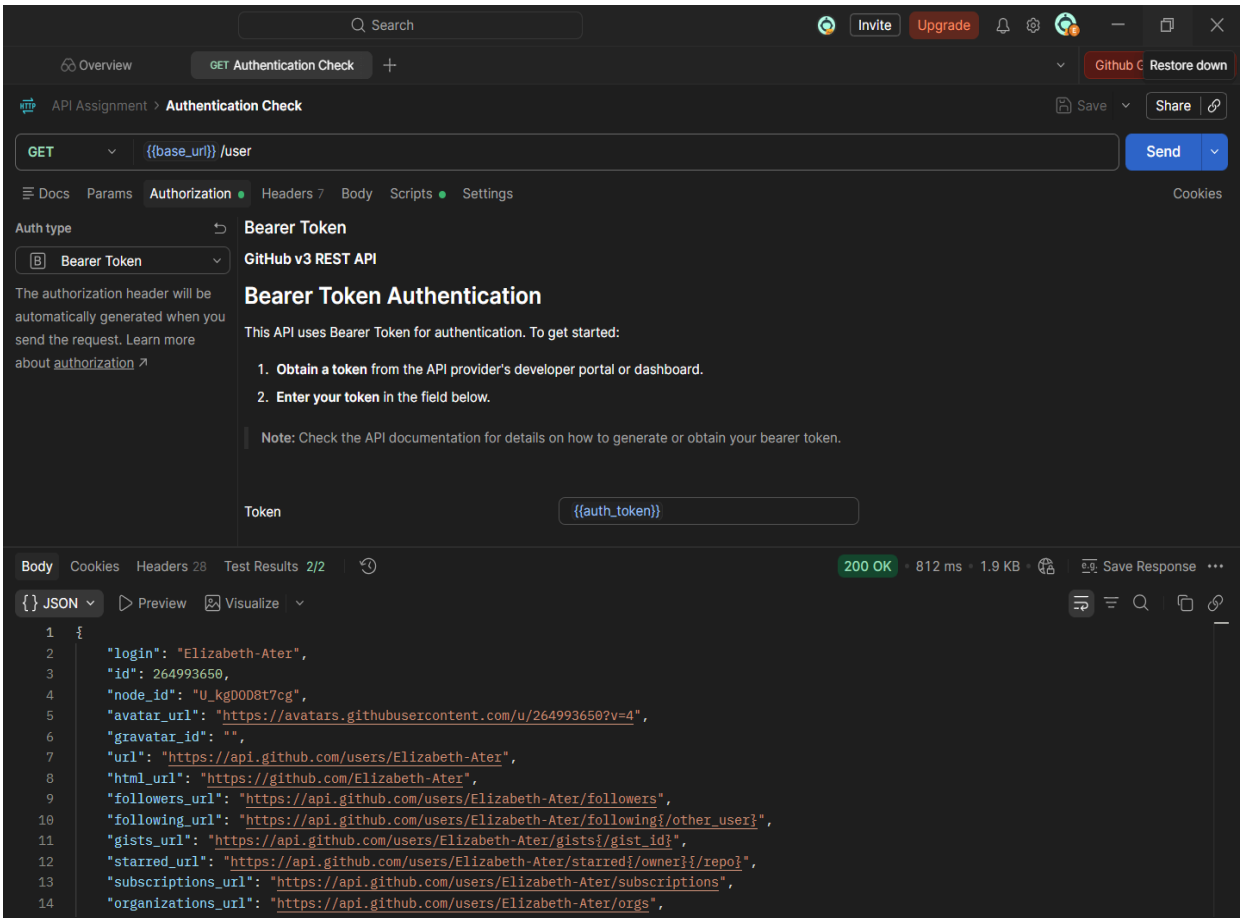
### Method and Url: GET



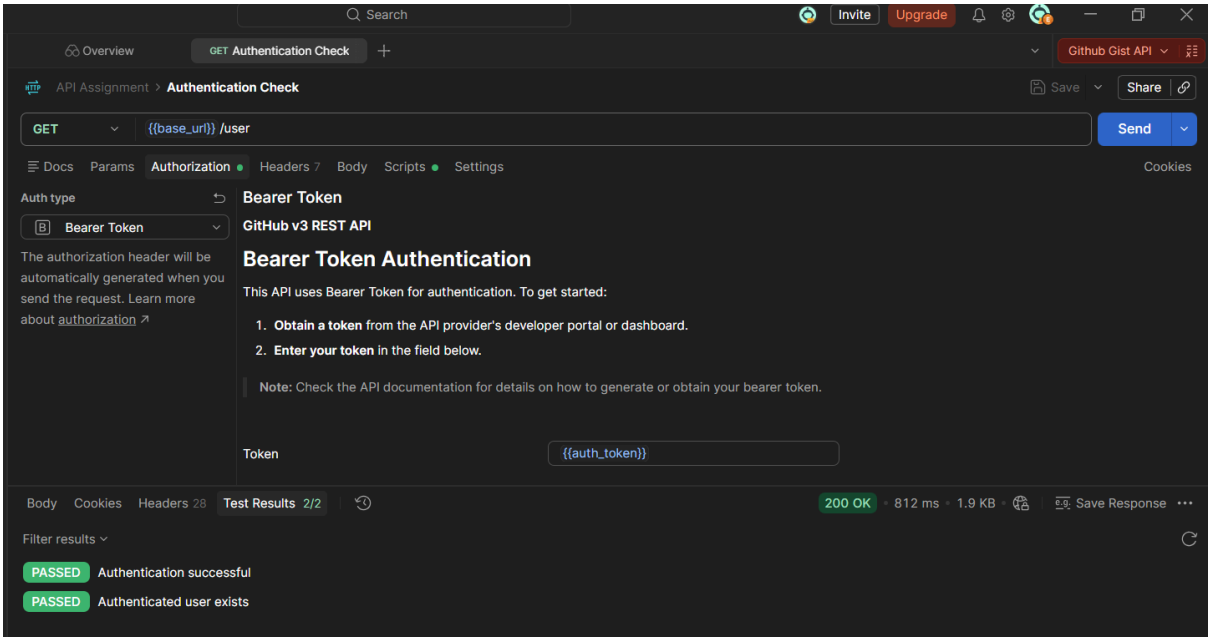
# Authorization




# Response



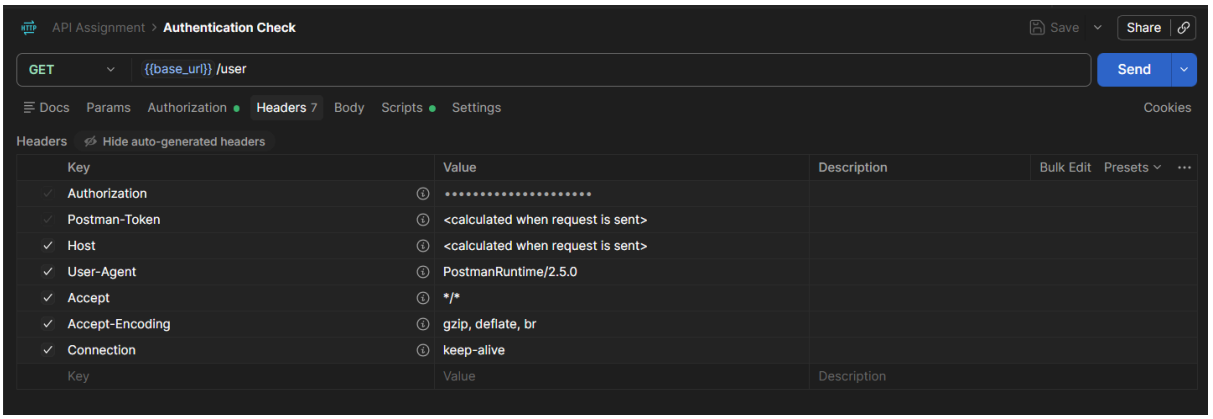
## Test

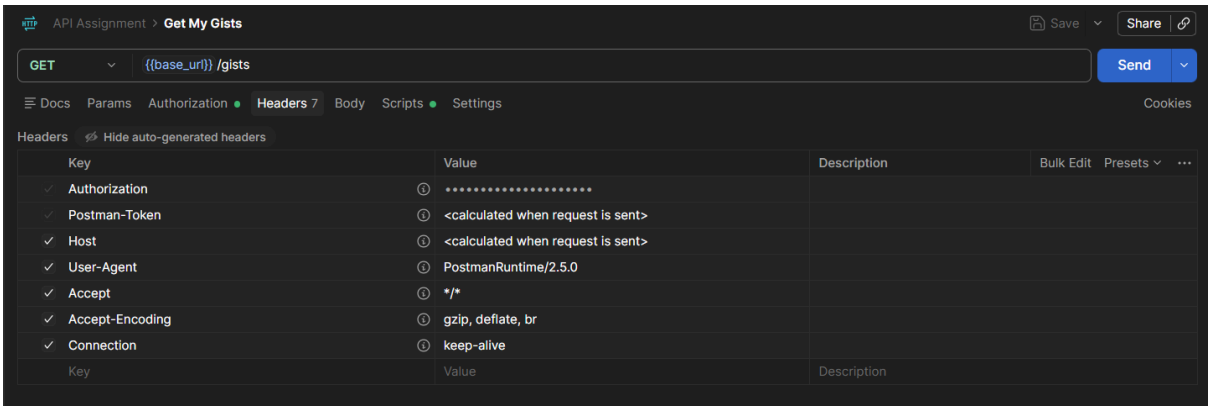


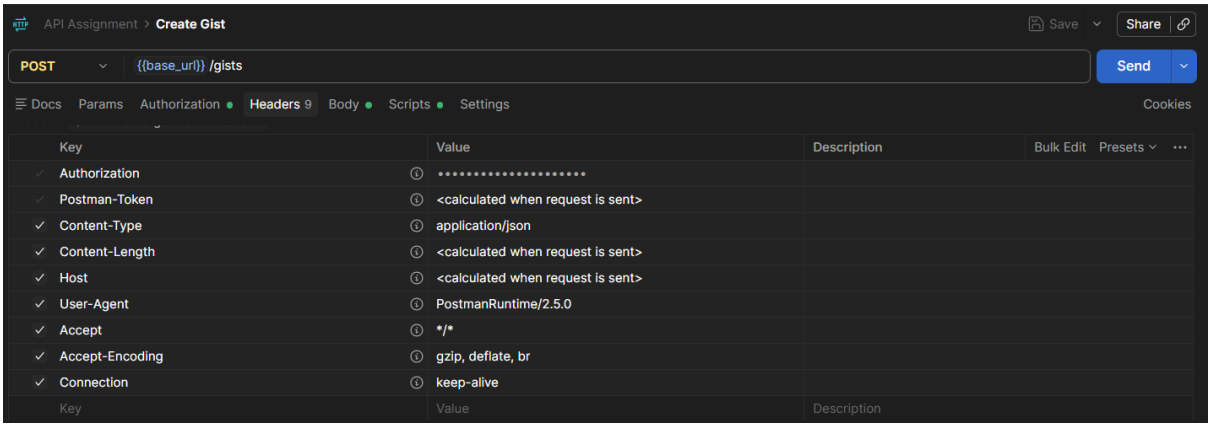

# Headers

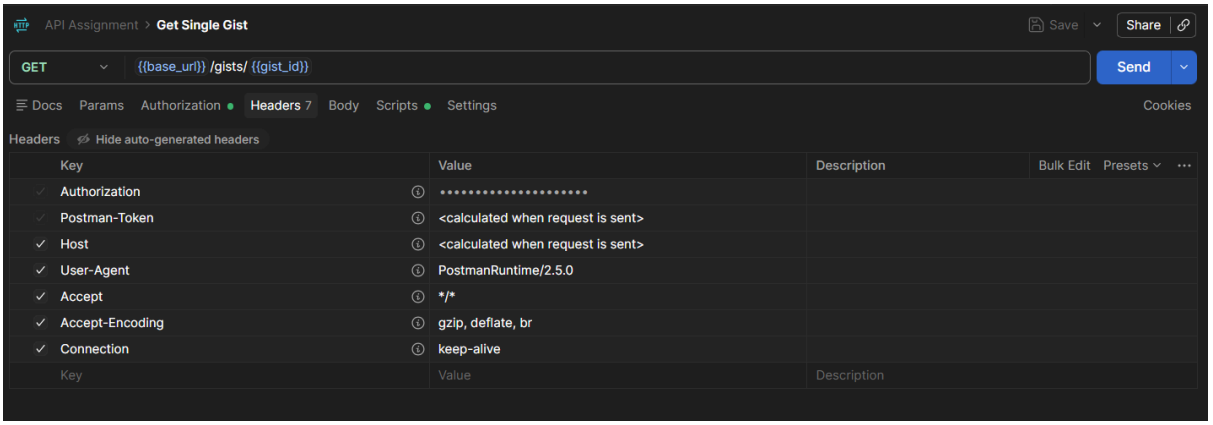
## Explanation

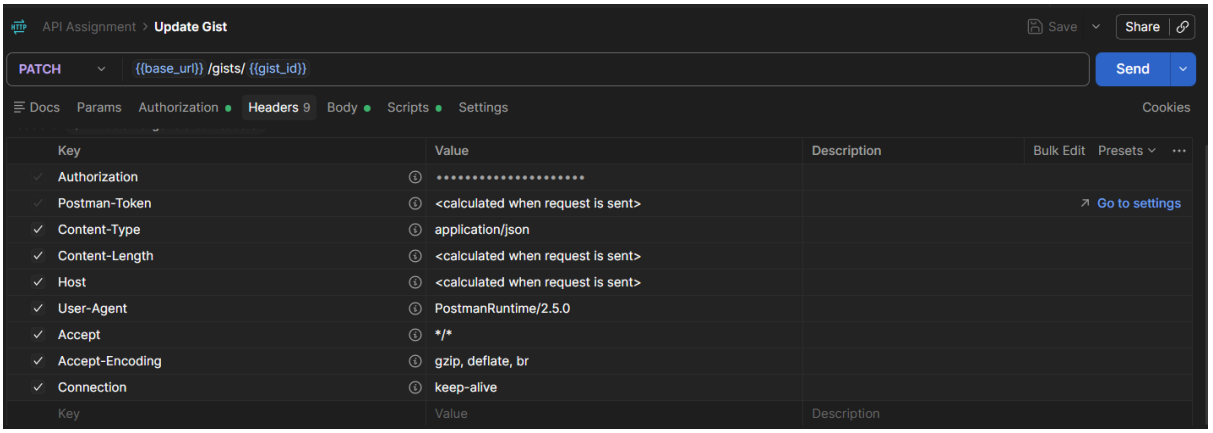
This section shows the Headers tab for each request. All requests include the Authorization header, which is generated automatically from the Bearer Token. The Create a Gist and Update a Gist requests also include a Content-Type header, since they send a JSON body.

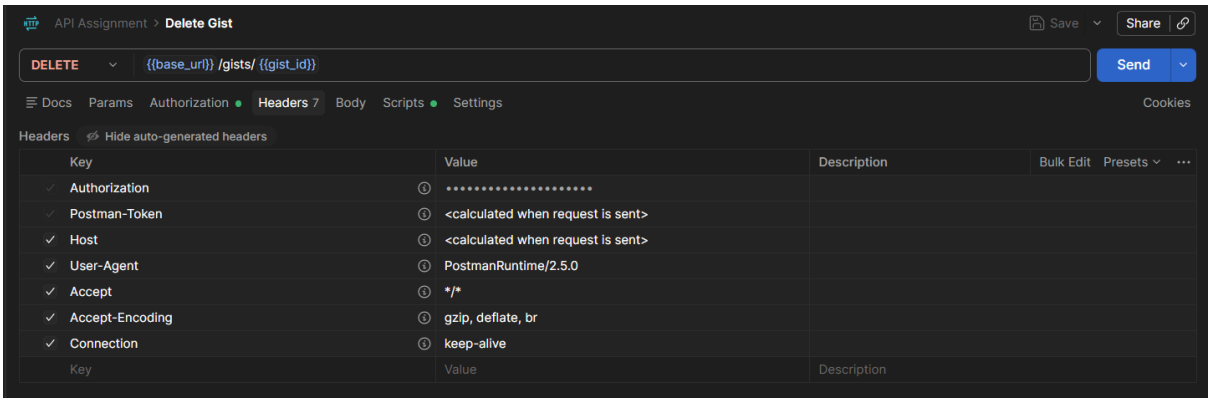



API Assignment > Create Gist
Save
Share
POST
{{base_url}} /gists
Send
Docs Params Authorization Headers 9 Body Scripts Settings
Cookies
Key Value Description Bulk Edit Presets
Authorization ••••••••••••••••••••
Postman-Token <calculated when request is sent>
Content-Type application/json
Content-Length <calculated when request is sent>
Host <calculated when request is sent>
User-Agent PostmanRuntime/2.5.0
Accept */*
Accept-Encoding gzip, deflate, br
Connection keep-alive
Key Value Description


API Assignment > Get Single Gist
Save
Share
GET
{{base_url}} /gists/ {{gist_id}}
Send
Docs Params Authorization Headers 7 Body Scripts Settings
Cookies
Headers Hide auto-generated headers
Key Value Description Bulk Edit Presets
Authorization ••••••••••••••••••••
Postman-Token <calculated when request is sent>
Host <calculated when request is sent>
User-Agent PostmanRuntime/2.5.0
Accept */*
Accept-Encoding gzip, deflate, br
Connection keep-alive
Key Value Description


API Assignment > Update Gist
Save
Share
PATCH
{{base_url}} /gists/ {{gist_id}}
Send
Docs Params Authorization Headers 9 Body Scripts Settings
Cookies
Key Value Description Bulk Edit Presets
Authorization ••••••••••••••••••••
Postman-Token <calculated when request is sent>
Go to settings
Content-Type application/json
Content-Length <calculated when request is sent>
Host <calculated when request is sent>
User-Agent PostmanRuntime/2.5.0
Accept */*
Accept-Encoding gzip, deflate, br
Connection keep-alive
Key Value Description

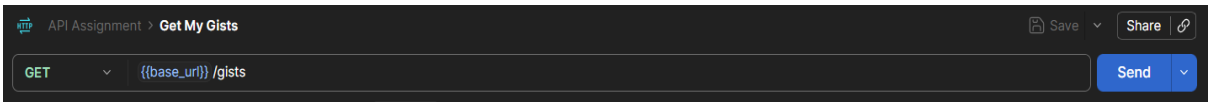

API Assignment > Delete Gist
Save
Share
DELETE
{{base_url}} /gists/ {{gist_id}}
Send
Docs Params Authorization Headers 7 Body Scripts Settings
Cookies
Headers Hide auto-generated headers
Key Value Description Bulk Edit Presets
Authorization ••••••••••••••••••••
Postman-Token <calculated when request is sent>
Host <calculated when request is sent>
User-Agent PostmanRuntime/2.5.0
Accept */*
Accept-Encoding gzip, deflate, br
Connection keep-alive
Key Value Description

# Get My Gists

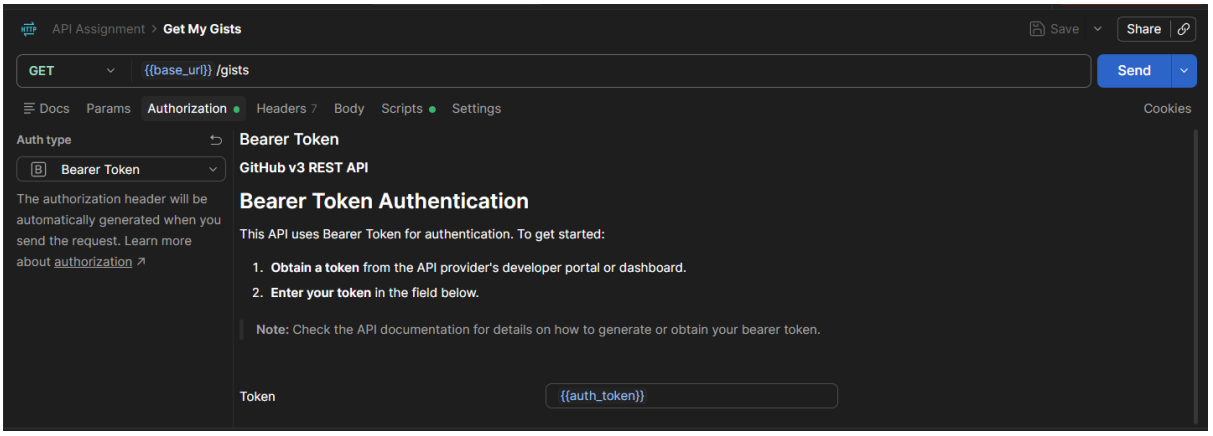
## Explanation

This request retrieves the authenticated user's Gists from GitHub. A successful response confirms that the request was processed correctly.

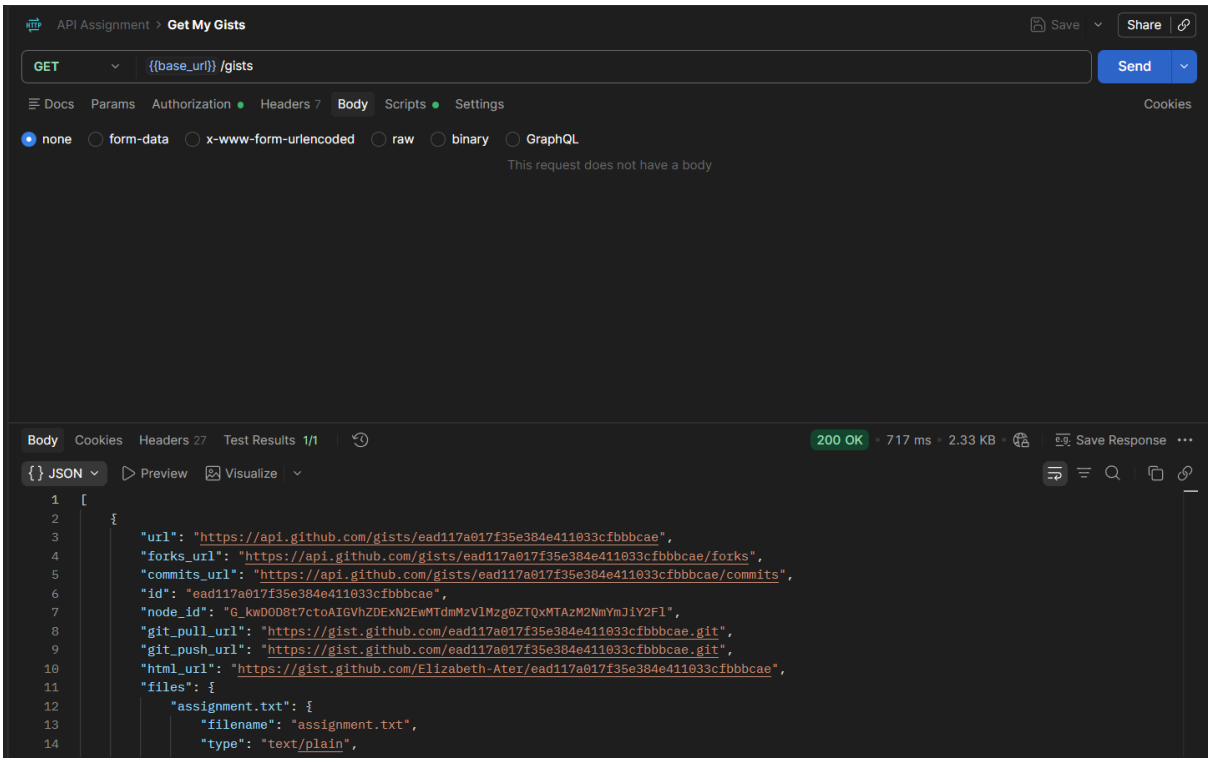
## Method and Url: GET



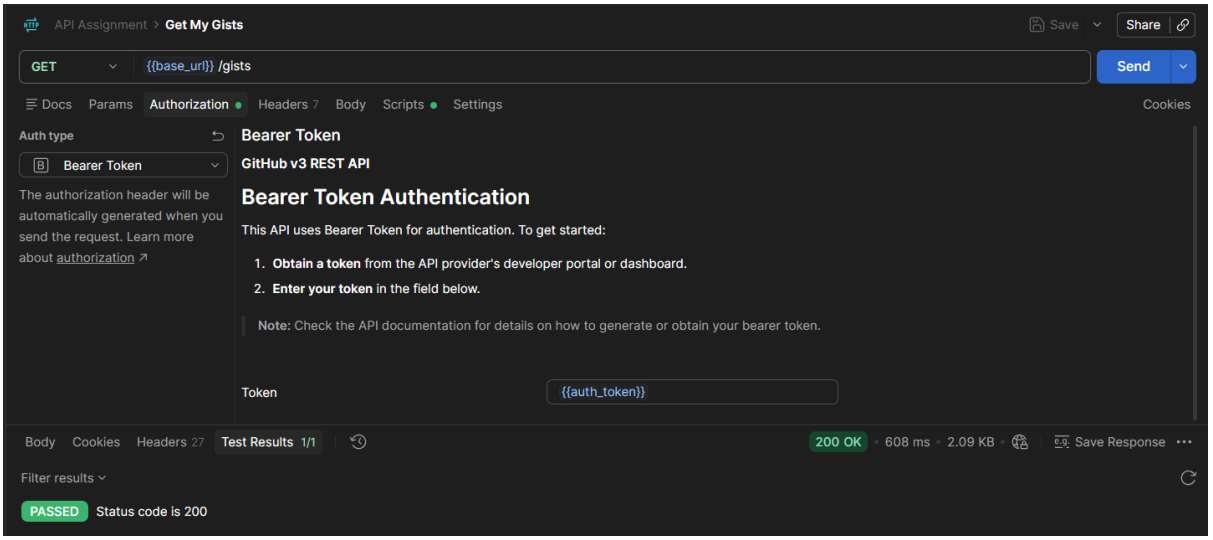

## Authorization




## Response



## Test



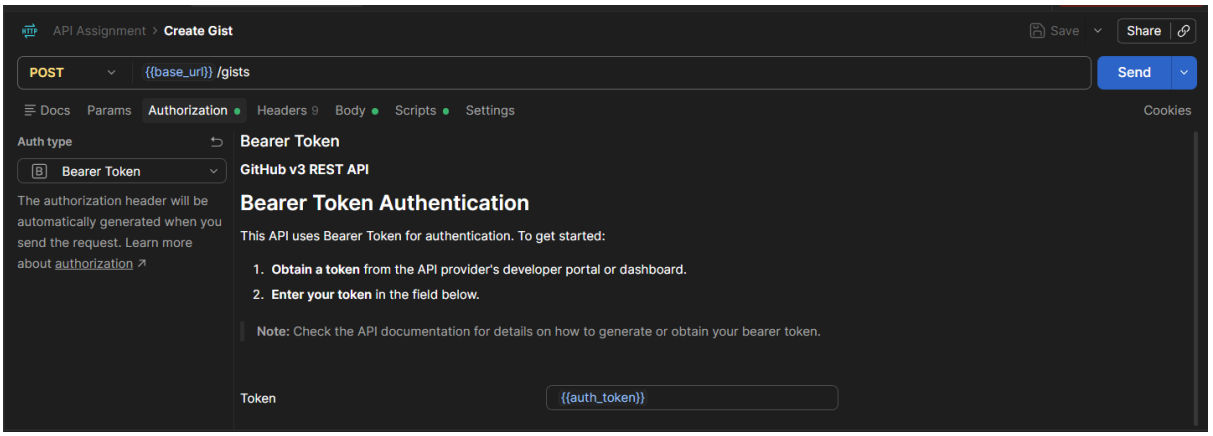

# Create a Gist

## Explanation

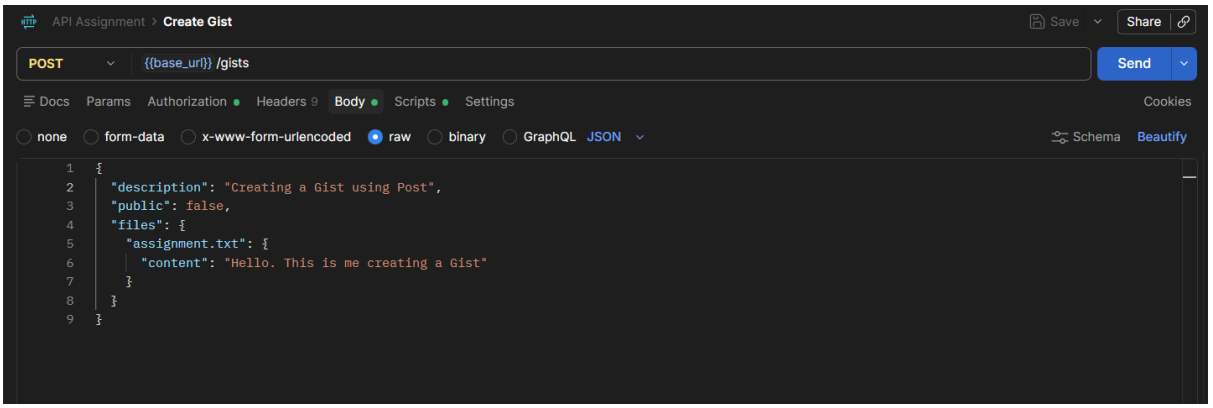
This request creates a new Gist. The response provides the Gist ID, which is stored in the gist_id environment variable for use in later requests.

## Method and Url: POST

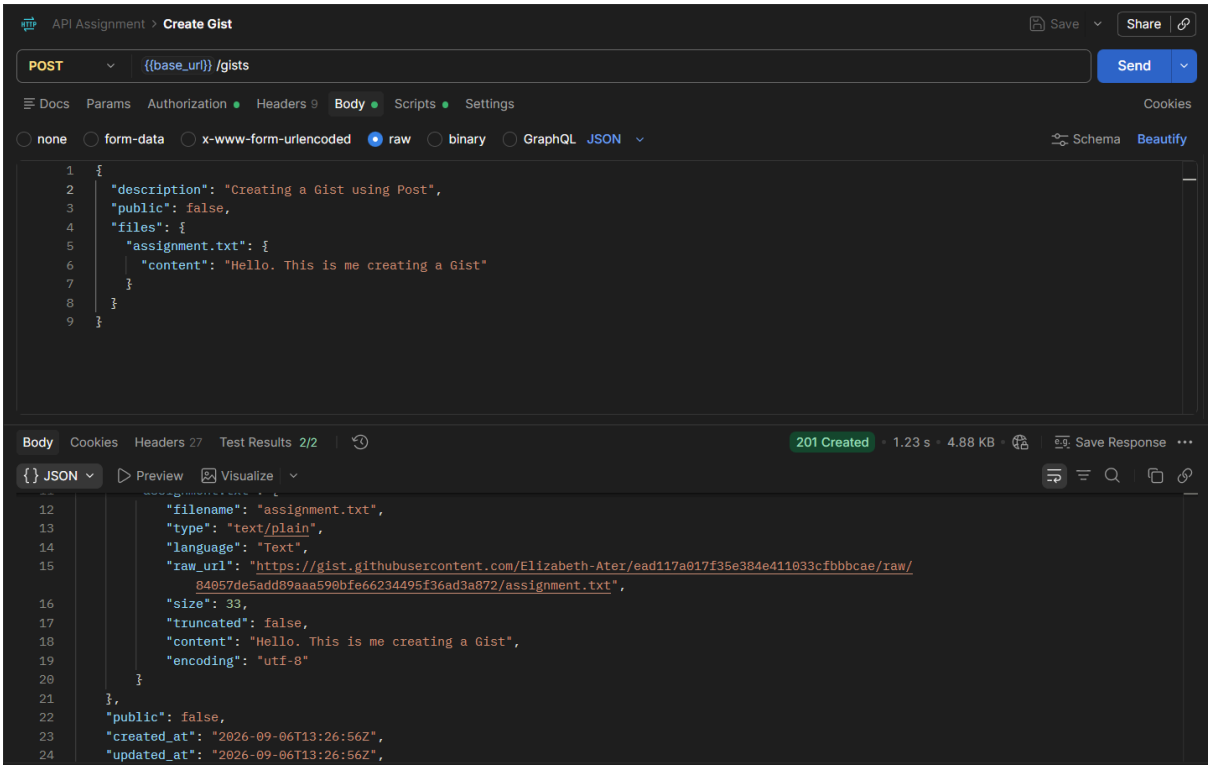

## Authorization



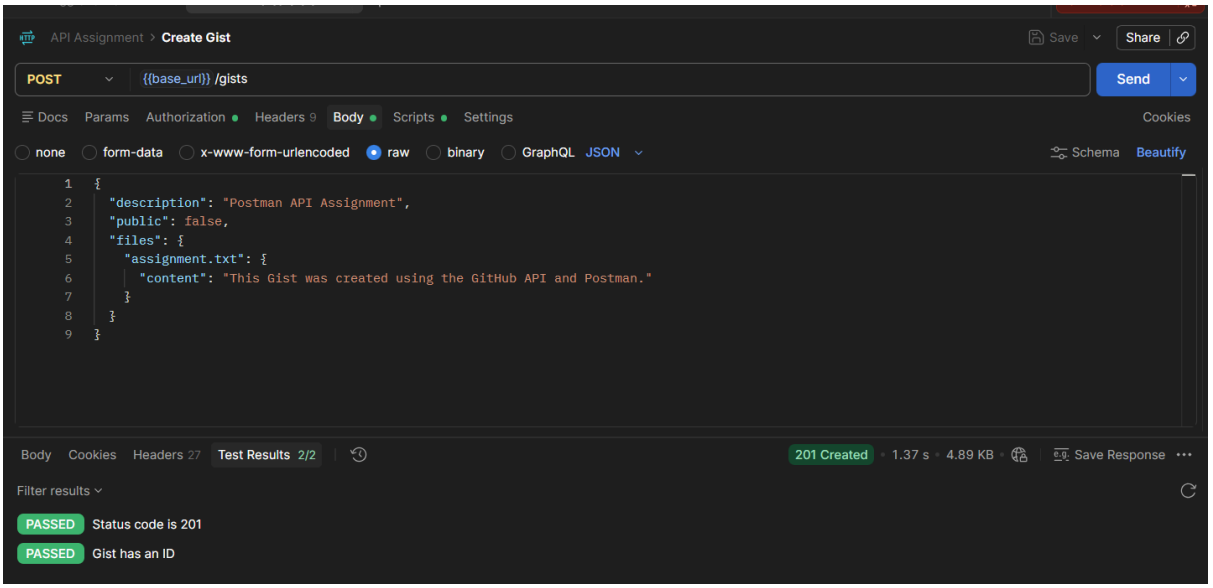
## Body




## Response



## Test

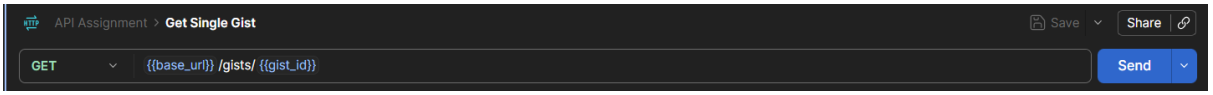



# Get a Single Gist

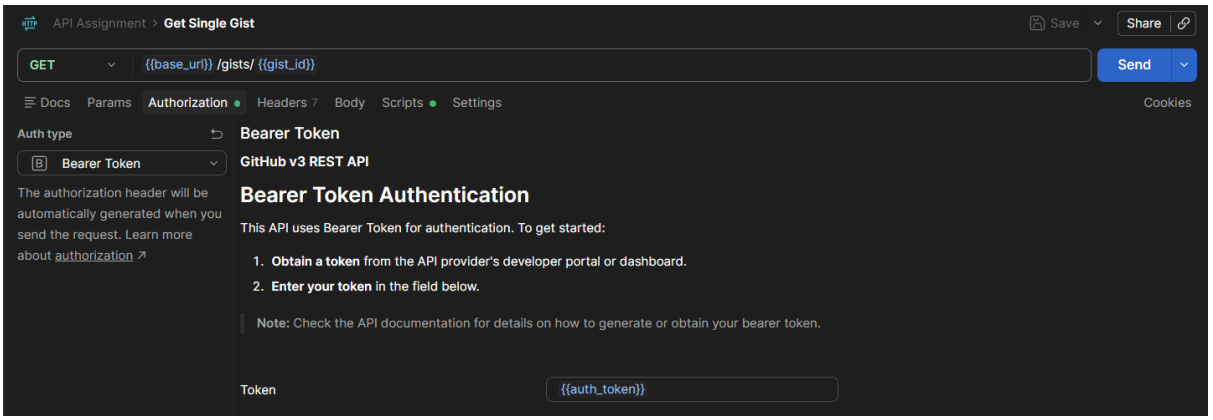
## Explanation

This request retrieves a specific Gist using the gist_id stored in the Postman environment.

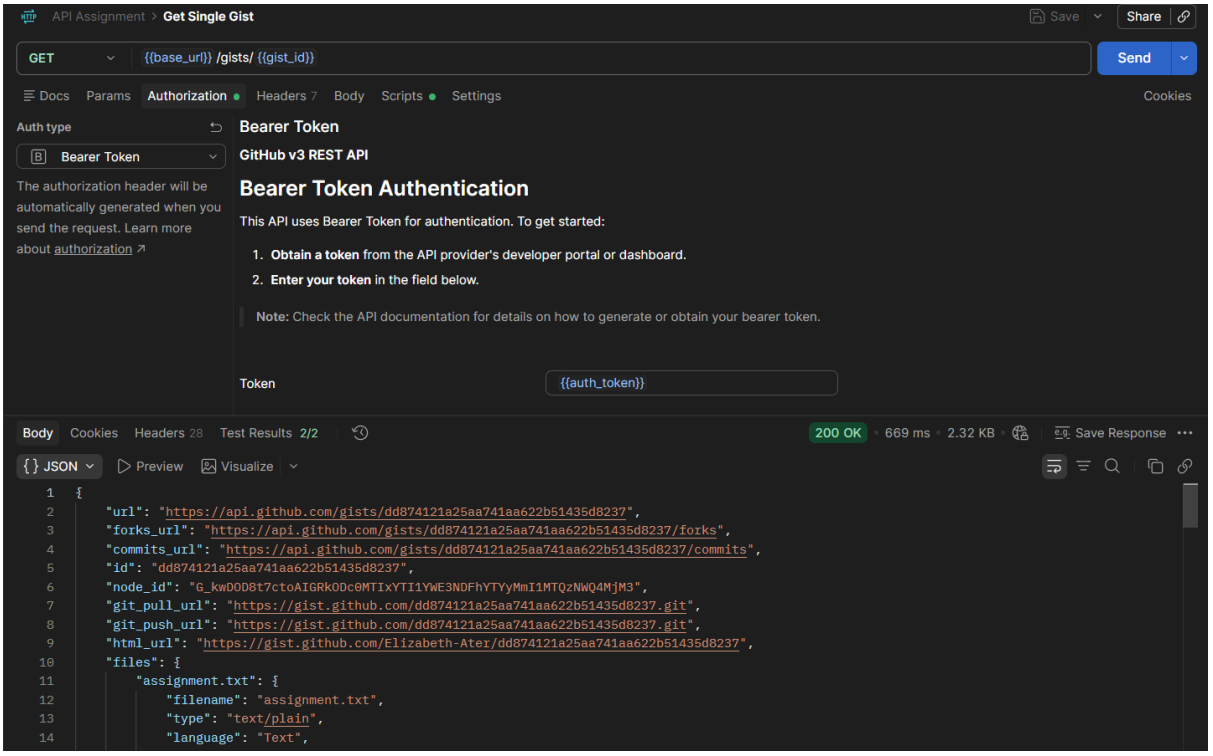
## Method and Url: GET



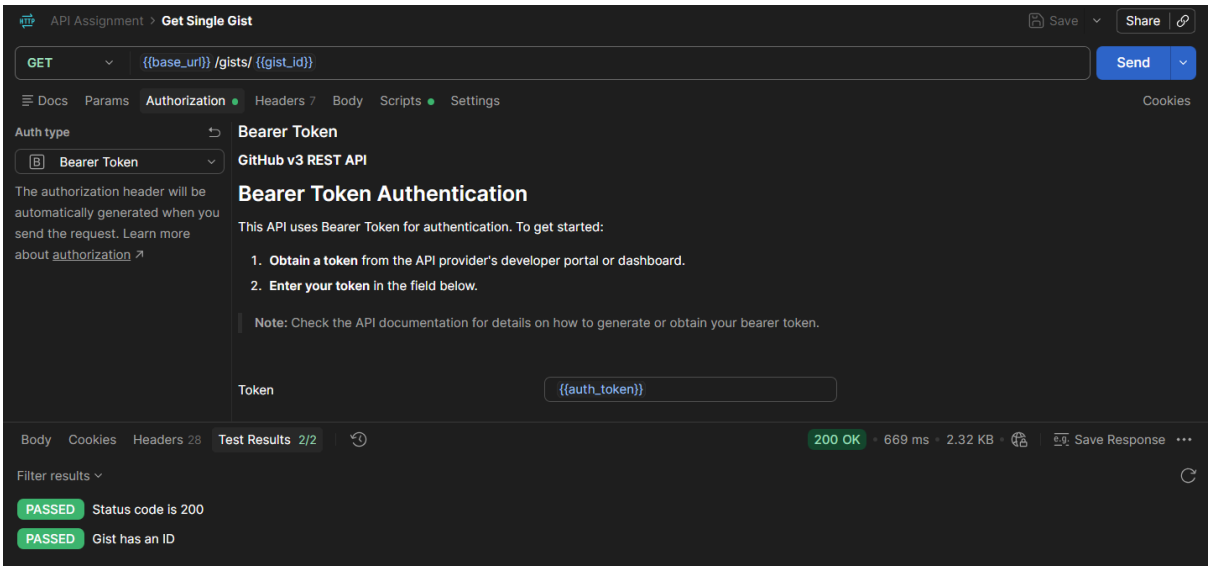

## Authorization



# Response




# Test



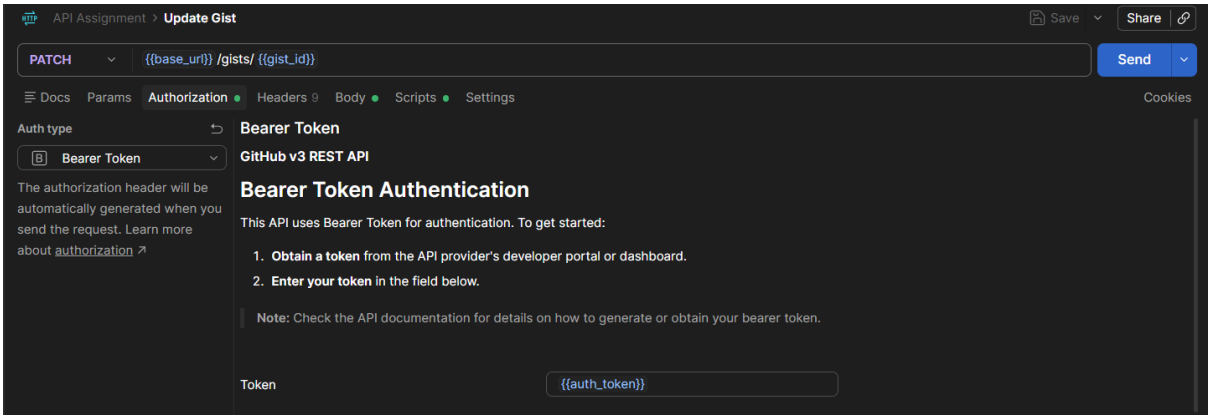
# Update a Gist

## Explanation

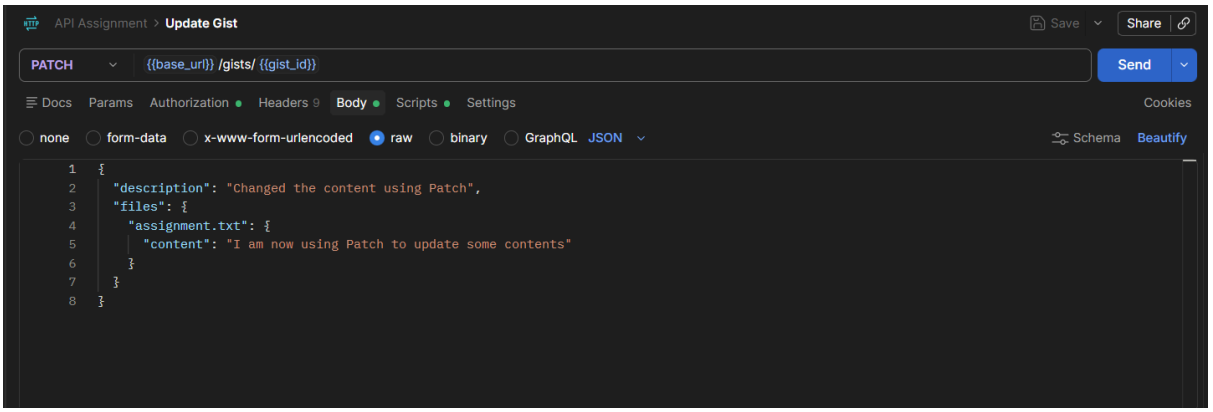
This request updates an existing Gist. The gist_id variable identifies the Gist that is being modified.

## Method and Url: PATCH

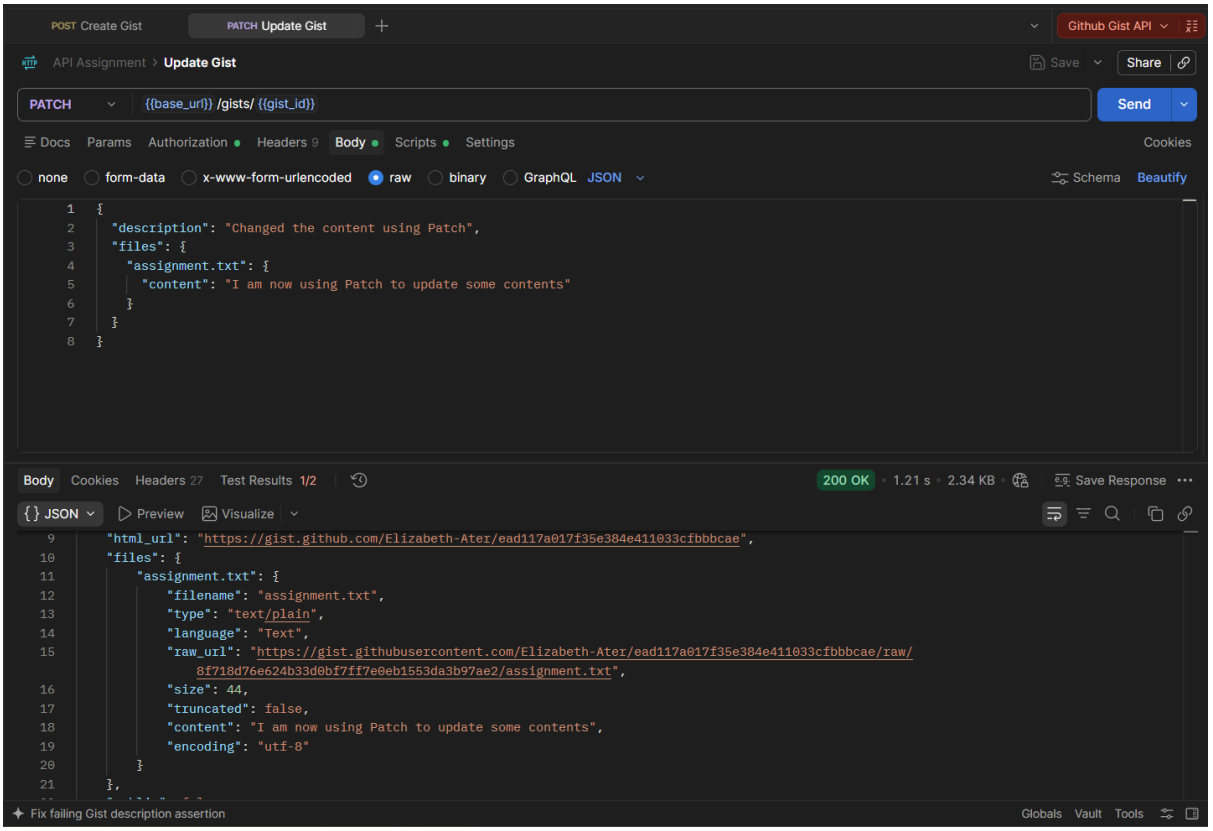

## Authorization



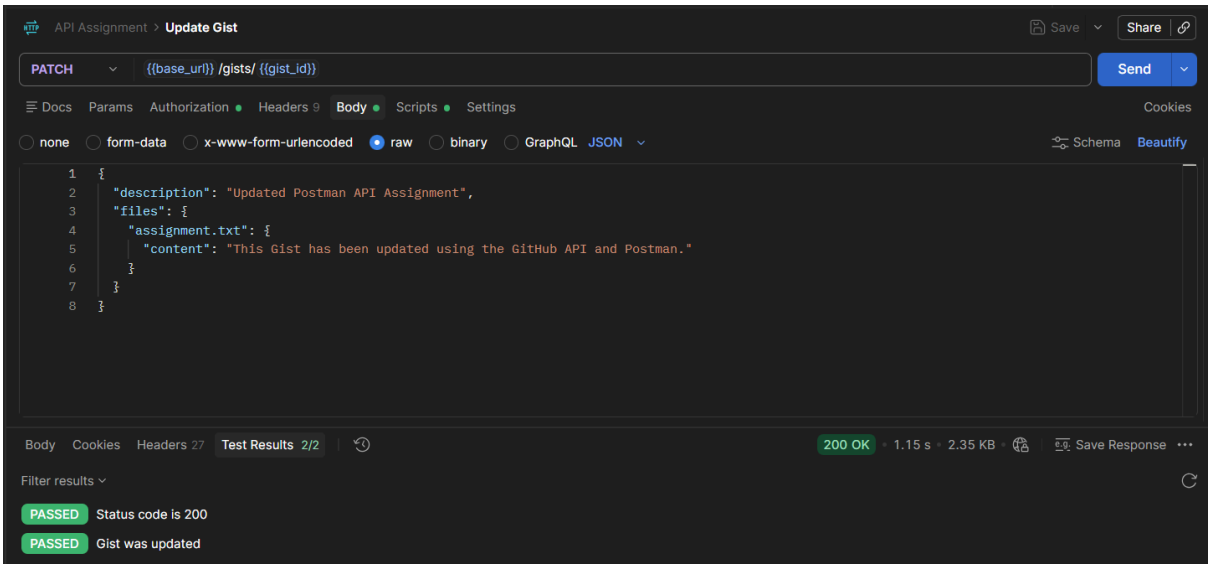

## Body



# Response




# Test



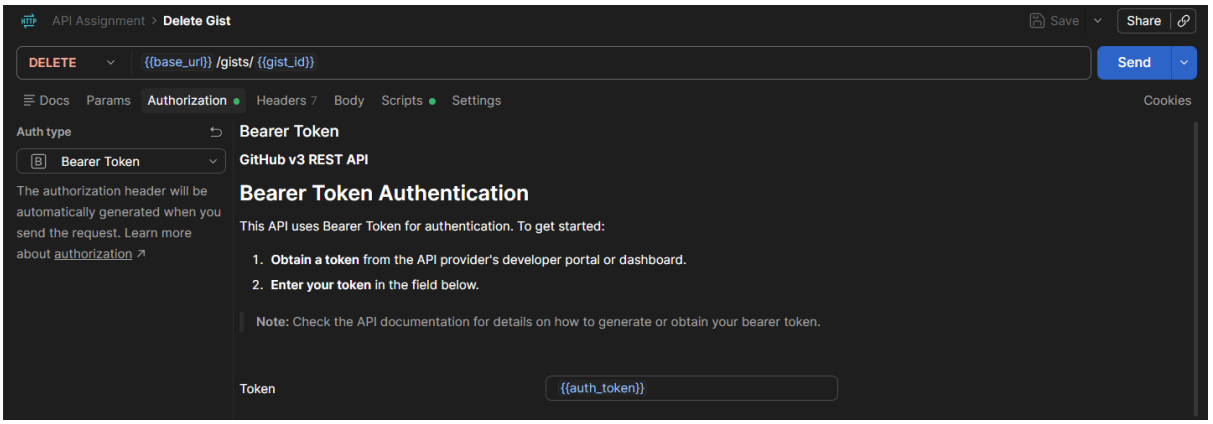
## Delete a Gist

### Explanation

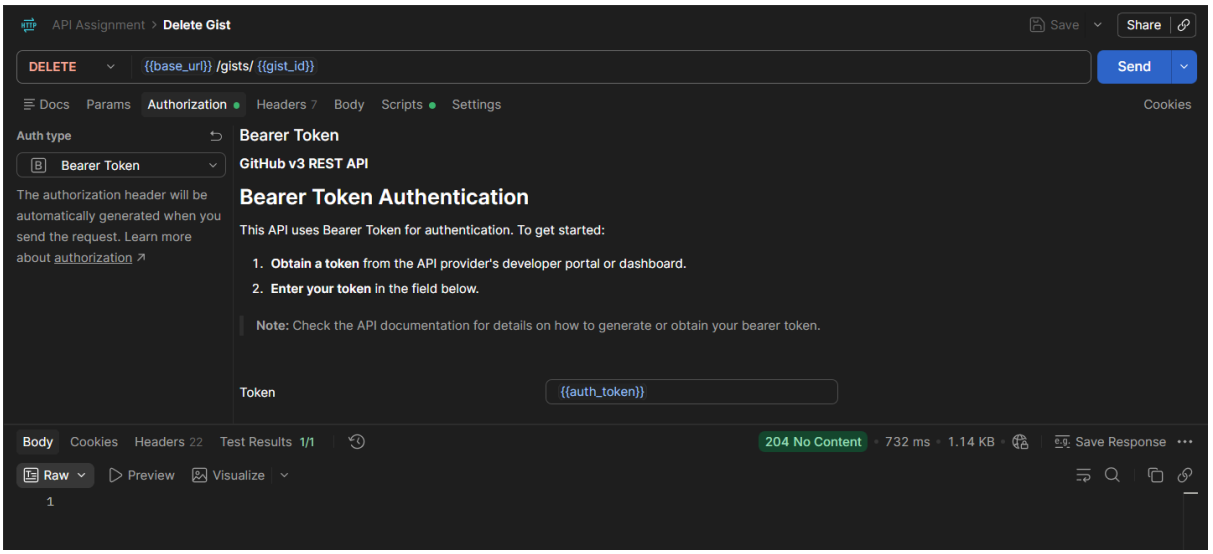
This request deletes the Gist identified by the gist_id variable. A successful deletion returns HTTP status code 204.

## Method and Url: DELETE

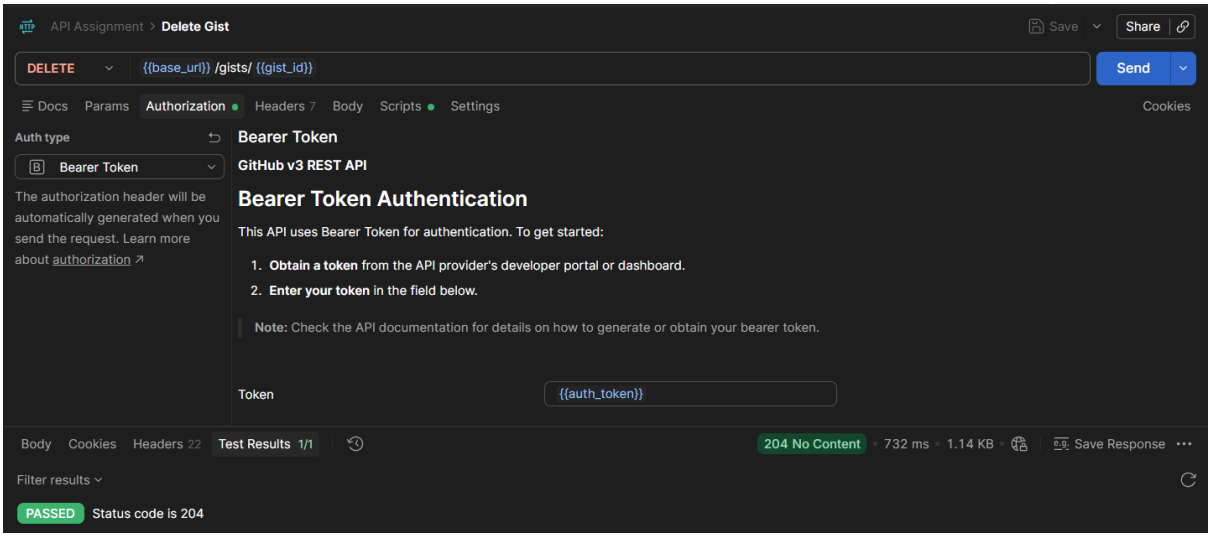

## Authorization




## Response



## Test

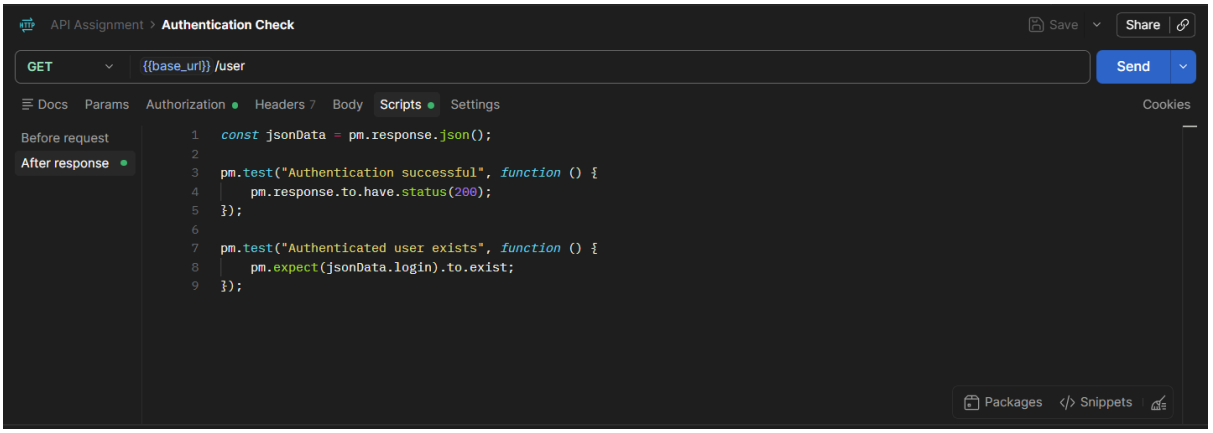


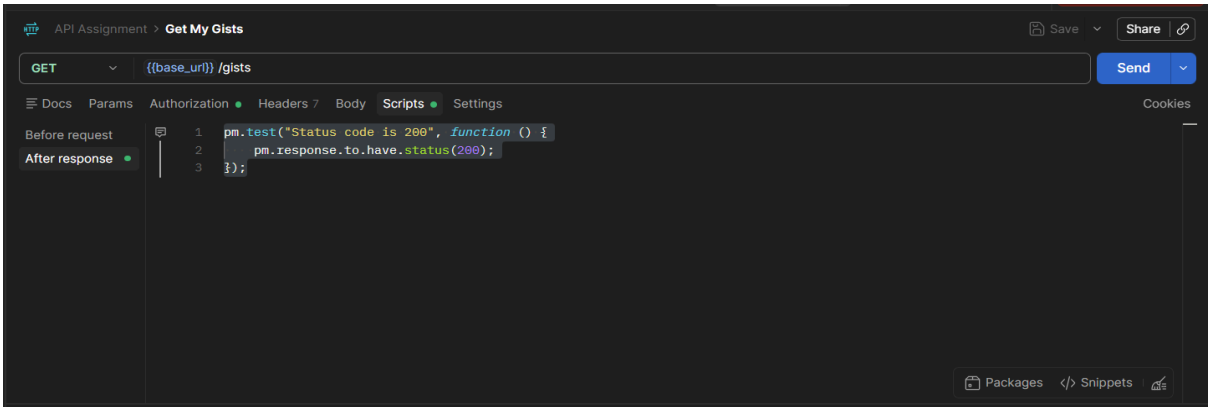

# Automated Testing

### Explanation

Postman test scripts were used to automatically verify the expected status codes and response data. This makes it possible to confirm that the API requests work correctly without checking every result manually.

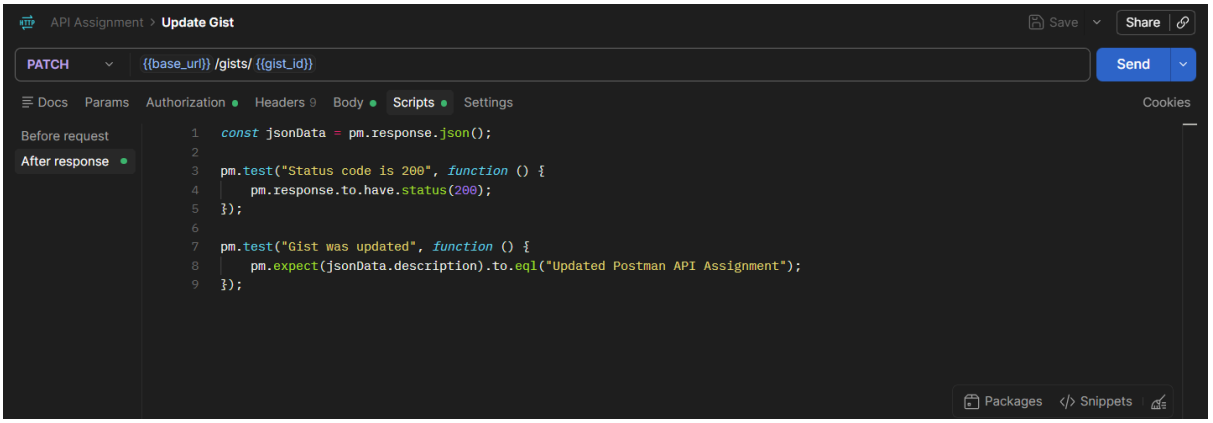
## Postman Tests

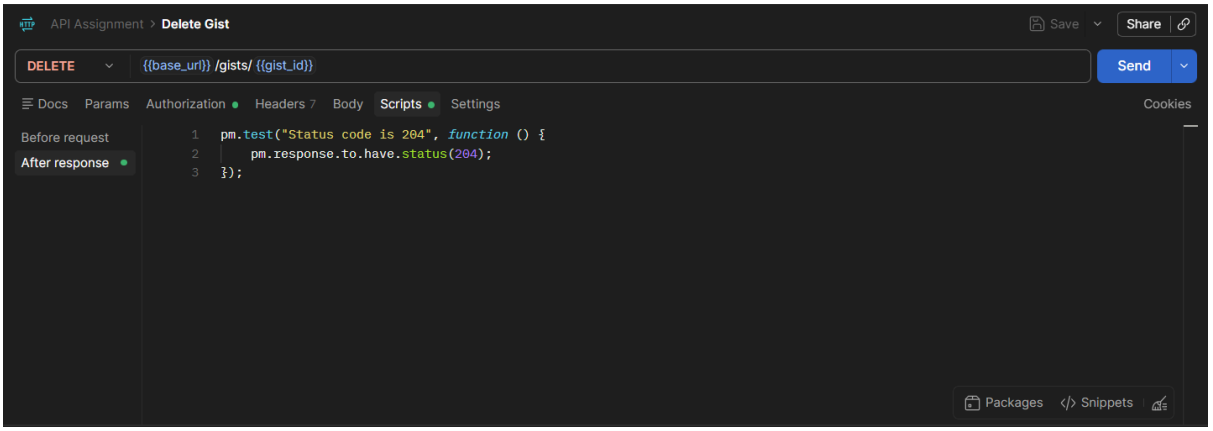



API Assignment > Create Gist
Save
Share
POST
{{base_url}} /gists
Send
Docs Params Authorization Headers 9 Body Scripts Settings
Cookies
Before request
After response
const jsonData = pm.response.json();
pm.environment.set("gist_id", jsonData.id);
pm.test("Status code is 201", function () {
pm.response.to.have.status(201);
});
pm.test("Gist has an ID", function () {
pm.expect(jsonData.id).to.exist;
});
Packages
Snippets


API Assignment > Get Single Gist
Save
Share
GET
{{base_url}} /gists/ {{gist_id}}
Send
Docs Params Authorization Headers 7 Body Scripts Settings
Cookies
Before request
After response
const jsonData = pm.response.json();
pm.test("Status code is 200", function () {
pm.response.to.have.status(200);
});
pm.test("Gist has an ID", function () {
pm.expect(jsonData.id).to.exist;
});
Packages
Snippets


API Assignment > Update Gist
Save
Share
PATCH
{{base_url}} /gists/ {{gist_id}}
Send
Docs Params Authorization Headers 9 Body Scripts Settings
Cookies
Before request
After response
const jsonData = pm.response.json();
pm.test("Status code is 200", function () {
pm.response.to.have.status(200);
});
pm.test("Gist was updated", function () {
pm.expect(jsonData.description).to.eql("Updated Postman API Assignment");
});
Packages
Snippets

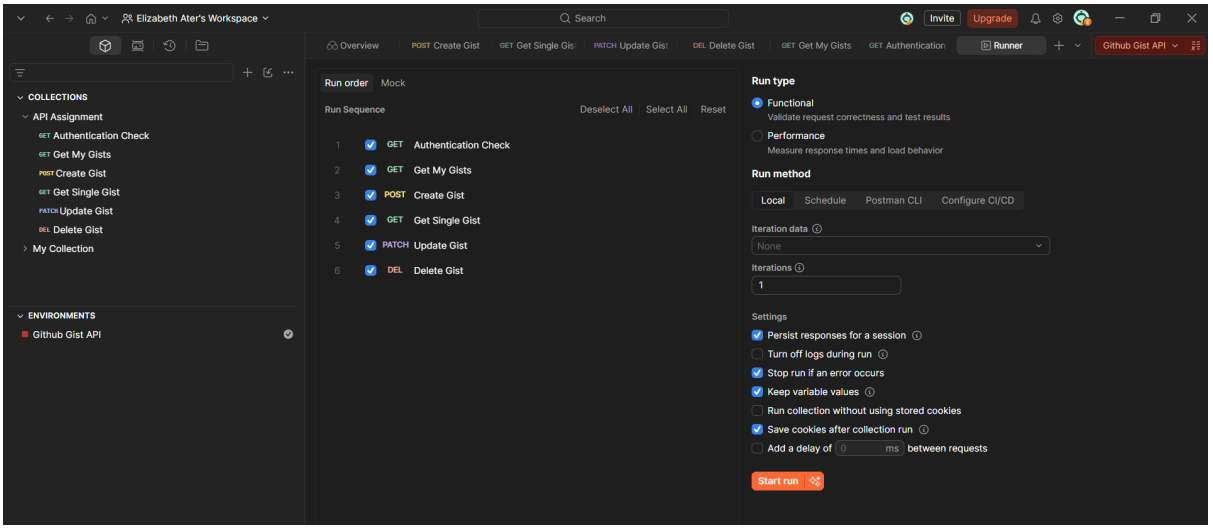

API Assignment > Delete Gist
Save
Share
DELETE
{{base_url}} /gists/ {{gist_id}}
Send
Docs Params Authorization Headers 7 Body Scripts Settings
Cookies
Before request
After response
pm.test("Status code is 204", function () {
pm.response.to.have.status(204);
});
Packages
Snippets

# Collection Runner

## Explanation

The Collection Runner was used to execute the API requests automatically in sequence. The successful results demonstrate that the requests and their automated tests are working correctly.

## Collection Runner Setup




## Requests Executed